

তোমার assignment টা মূলত **Factory Method Design Pattern** বুঝে existing code refactor করা নিয়ে।
তুমি যেহেতু Java জানো এবং OOP করছো, তাই এটা actually খুব important একটা concept।

আমি একদম basic থেকে বাংলায় explain করছি।

Design Pattern আসলে কী?

Design Pattern হলো programming এর common problem solve করার tested solution।

মানে:

“এই ধরনের problem এ software engineer রা সাধারণত এভাবে code organize করে।”

এটা কোনো library না, কোনো framework না।

এটা coding structure / architecture idea।

তোমাদের বর্তমান problem কোথায়?

এখন system এ আছে:

- Bus
- Taxi
- MotorcycleDelivery
- ElectricScooter

সবগুলো **Vehicle** interface implement করছে।

এটা ভালো।

কিন্তু problem হচ্ছে:

যদি client code এ everywhere এভাবে object create করা হয়:

```
Vehicle v1 = new Bus();  
Vehicle v2 = new Taxi();  
Vehicle v3 = new ElectricScooter();
```

তাহলে future এ problem হবে।

কেন **problem** হবে?

ধরো পরে নতুন vehicle আসলো:

- Train
- DroneDelivery
- Ambulance

তাহলে তোমাকে সব জায়গায় গিয়ে modify করতে হবে:

```
if(type.equals("bus"))  
    v = new Bus();
```

```
else if(type.equals("taxi"))  
    v = new Taxi();
```

```
else if(type.equals("drone"))  
    v = new DroneDelivery();
```

এটাকে বলে:

Tight Coupling

মানে client code concrete class সম্পর্কে directly জানে।

এটা bad design কারণ:

- maintain করা কঠিন
- extend করা কঠিন
- reusable না
- scalable না

Factory Method Pattern কী solve করে?

Factory Method object creation logic কে আলাদা করে দেয়।

মানে:

Object create করার দায়িত্ব একটা Factory class নিবে।

Client শুধু বলবে:

আমাকে একটা Taxi দাও

Factory internally decide করবে কোন object create হবে।

Real Life Example

ধরো তুমি restaurant এ গেলে।

তুমি chef কে directly বলো না:

- oil আনো
- chicken আনো
- fry করো

তুমি শুধু বলো:

“একটা chicken burger দিন।”

Restaurant internally তৈরি করে দেয়।

এটাই Factory Pattern।

তোমাদের Assignment এ কী হবে?

বর্তমানে:

```
Vehicle v = new Bus();
```

এভাবে object create হচ্ছে।

এখন এটা replace করে করতে হবে:

```
Vehicle v = VehicleFactory.createVehicle("bus");
```

এখন Factory decide করবে কোন object বানাবে।

Factory Pattern Structure

Factory Method এ সাধারণত থাকে:

1. Product Interface

তোমাদের code এ:

```
interface Vehicle
```

এটাই Product interface।

2. Concrete Products

Bus

Taxi

MotorcycleDelivery

ElectricScooter

এগুলো concrete product।

3. Factory Class

নতুন class বানাতে হবে:

```
class VehicleFactory
```

এটা object create করবে।

Factory Class কেমন হবে?

Example:

```
class VehicleFactory {  
  
    public static Vehicle createVehicle(String type) {  
  
        if(type.equalsIgnoreCase("bus")) {  
            return new Bus();  
        }  
  
        else if(type.equalsIgnoreCase("taxi")) {  
            return new Taxi();  
        }  
  
        else if(type.equalsIgnoreCase("motorcycle")) {  
            return new MotorcycleDelivery();  
        }  
  
        else if(type.equalsIgnoreCase("scooter")) {  
            return new ElectricScooter();  
        }  
  
        return null;  
    }  
}
```

Client Code কেমন হবে?

আগে:

```
Vehicle v = new Bus();
```

এখন:

```
Vehicle v = VehicleFactory.createVehicle("bus");
```

এখানে লাভ কী হলো?

Client এখন জানেই না **Bus** class কী।

সে শুধু factory use করছে।

এখন future এ নতুন class add করলে:

class DroneDelivery implements Vehicle

শুধু Factory modify করলেই হবে।

Client code change লাগবে না।

Assignment এ সম্ভবত কী করতে হবে?

তোমার teacher likely চাইবে:

Step 1

Existing Vehicle interface keep করবে।

Step 2

সব concrete class keep করবে।

- Bus
 - Taxi
 - MotorcycleDelivery
 - ElectricScooter
-

Step 3

নতুন Factory class বানাবে।

VehicleFactory

Step 4

সব `new Bus()` remove করবে।

Replace করবে:

`VehicleFactory.createVehicle()`

দিয়ে।

একটা **Full Flow** দেখো

Without Factory

`Vehicle v = new Bus();`

`v.startTrip();`

Client directly Bus জানে।

With Factory

`Vehicle v = VehicleFactory.createVehicle("bus");`

`v.startTrip();`

এখন client শুধু Vehicle জানে।

Concrete class hidden.

Assignment এর মূল learning objective

তোমাদের teacher আসলে শেখাতে চাচ্ছে:

“Encapsulation of Object Creation”

মানে object creation logic আলাদা রাখা।

Factory Method Pattern এর Main Advantages

1. Loose Coupling

Client concrete class জানে না।

2. Easy Maintenance

সব object creation এক জায়গায়।

3. Easy Extension

নতুন vehicle add করা সহজ।

4. Cleaner Code

if-else everywhere লাগে না।

তোমাদের **code** এ আর কী **improve** করা যায়?

বর্তমান code এ duplication আছে:

সব class এ:

capacity
farePerKm
route

এগুলো repeated।

Future এ abstract class use করা যায়:

abstract class BaseVehicle

But তোমাদের current assignment probably শুধু Factory Method নিয়ে।

Factory Method vs Simple Factory

Technically:

VehicleFactory.createVehicle()

এটা অনেক teacher “Factory Method” বললেও actually এটা অনেক সময় “Simple Factory” style।

True Factory Method এ creator inheritance থাকে।

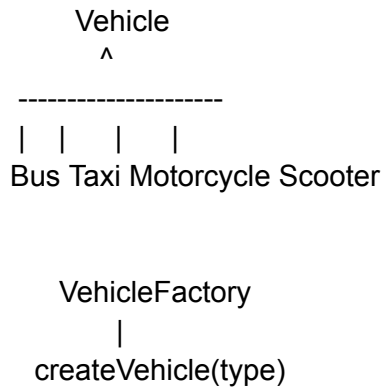
But university lab এ usually এই version-ই accept করে।

তোমার **Assignment** এ সম্ভবত **Report** এ যা লিখবে

তুমি লিখতে পারো:

- Problem in current system
 - Tight coupling issue
 - Need for scalability
 - Factory Method solution
 - Benefits
 - UML Diagram
 - Refactored code
-

UML Idea



খুব সহজ ভাষায় Final Summary

আগে:

```
new Bus()
new Taxi()
```

সব জায়গায় manually object তৈরি হচ্ছিল।

এখন:

```
VehicleFactory.createVehicle()
```

একটা centralized factory object তৈরি করবে।

এটাই Factory Method Pattern এর মূল idea।